

第 36 讲

Learning Rate 学习率

为什么这个数字太大训练会爆炸，太小又学不动

《面向 LLM / Agent Systems 科研的深度学习零基础课》

本讲定位

第 33 讲已经知道 Gradient Descent 的基本更新式：

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

第 34–35 讲又加入了 Mini-Batch、Gradient Noise、Optimizer Step、Effective Batch 等训练时间尺度。本讲只聚焦公式里看似最不起眼、实际最敏感的一个量：**Learning Rate (学习率)** η 。我们要真正理解它如何控制参数更新步长、为什么会出现“太小学不动、太大直接炸掉”、为什么不同曲率方向会限制可用学习率，以及为什么同一个数值在 SGD 和 Adam 中不能机械类比。

学习目标

学完本讲，你应该能够：

1. 用自己的话解释 Learning Rate 是什么、为什么 Gradient 本身不能决定完整更新；
2. 看到 $\theta \leftarrow \theta - \eta g$ 时，能够逐项解释 θ 、 g 、 η 、 $\Delta\theta$ ；
3. 用一个一维二次函数手算不同学习率下的参数轨迹；
4. 理解“学习率太大”为什么可能振荡、发散，而不是简单地“下降更快”；
5. 理解 Loss Landscape 的曲率为什么会限制稳定学习率；
6. 区分 **Gradient Norm**、**Update Norm**、**Learning Rate** 三者；
7. 知道 Batch Size、Gradient Noise 与 Learning Rate 存在耦合，但没有一条可以机械套用的固定比例公式；
8. 理解为什么 SGD 与 Adam 的 nominal learning rate 不能直接按数值大小比较；
9. 理解 Warmup、Constant LR、Decay、Cosine Schedule 分别在改变什么；
10. 看训练曲线时初步判断 LR 可能过大、过小，还是问题根本不在 LR；
11. 在 LLM / Transformer / Agent Systems 论文中正确读取 peak LR、warmup steps、scheduler、global batch 与 optimizer step。

1 Learning Rate 到底是什么？

Gradient 告诉我们当前参数空间中 Loss 上升最快的局部方向：

$$g_t = \nabla L(\theta_t)$$

因此最基本的 Gradient Descent 沿反方向移动：

$$\theta_{t+1} = \theta_t - \eta g_t$$

这里：

- θ_t ：当前参数；
- g_t ：当前参数处的 Gradient；
- η ：Learning Rate；
- $-\eta g_t$ ：这一步真正施加到参数上的更新量 $\Delta\theta$ 。

所以最基本的 SGD 情况下：

$$\Delta\theta = -\eta g$$

原理

Gradient 主要回答“往哪个方向、局部坡度多大”；Learning Rate 主要控制“根据这个局部信息，实际迈多大一步”。如果只有 Gradient 而没有步长，算法仍然不知道参数究竟应该改多少。

可以把它类比成下山：Gradient 像是脚下测到的坡度和方向，而 Learning Rate 像每一步跨出的距离。方向正确并不意味着一步跨得越大越好，因为你看到的只是当前位置附近的地形。

2 一个数字例子：Learning Rate 怎样改变一次更新？

考虑：

$$L(w) = (w - 3)^2$$

当前：

$$w = 8$$

则：

$$L = (8 - 3)^2 = 25$$

导数：

$$\frac{dL}{dw} = 2(w - 3) = 10$$

如果学习率：

$$\eta = 0.01$$

那么：

$$w_{new} = 8 - 0.01 \times 10 = 7.9$$

如果：

$$\eta = 0.1$$

则：

$$w_{new} = 8 - 0.1 \times 10 = 7$$

如果：

$$\eta = 0.5$$

则：

$$w_{new} = 8 - 0.5 \times 10 = 3$$

恰好一步到最低点。

同一个 Gradient, 不同 Learning Rate

当前 Gradient 都是 10, 但：

$$\eta = 0.01 \Rightarrow \Delta w = -0.1$$

$$\eta = 0.1 \Rightarrow \Delta w = -1$$

$$\eta = 0.5 \Rightarrow \Delta w = -5$$

所以 Learning Rate 不是在改变 “Gradient 是多少”，而是在改变**我们根据这个 Gradient 真正更新多少参数**。

3 为什么 Learning Rate 太大会失败？——局部信息不是全局地图

第 20–21 讲已经强调：Gradient 是当前点的**局部信息**。它可以通过一阶近似理解：

$$L(\theta + \Delta\theta) \approx L(\theta) + \nabla L(\theta)^T \Delta\theta$$

取：

$$\Delta\theta = -\eta \nabla L$$

得到：

$$L(\theta + \Delta\theta) - L(\theta) \approx -\eta \|\nabla L\|^2$$

对于足够小的正 η , 右边通常为负, 因此 Loss 会下降。

但是注意关键词：**足够小、局部近似**。

常见误区

“负梯度是下降方向” 不等于 “沿负梯度走任意远都一定下降”。当一步太大, 你可能已经离开了当前 Gradient 能够准确描述的局部区域, 甚至直接跨过山谷最低点, 跑到另一边更高的位置。

4 二次函数上可以精确看见收敛、振荡与发散

仍考虑：

$$L(w) = (w - 3)^2$$

令误差：

$$e_t = w_t - 3$$

Gradient：

$$\frac{dL}{dw} = 2e_t$$

更新：

$$w_{t+1} = w_t - 2\eta e_t$$

因此：

$$e_{t+1} = (1 - 2\eta)e_t$$

这条式子直接告诉我们每一步误差会乘上一个固定因子：

$$r = 1 - 2\eta$$

如果希望误差越来越小，必须：

$$|r| < 1$$

也就是：

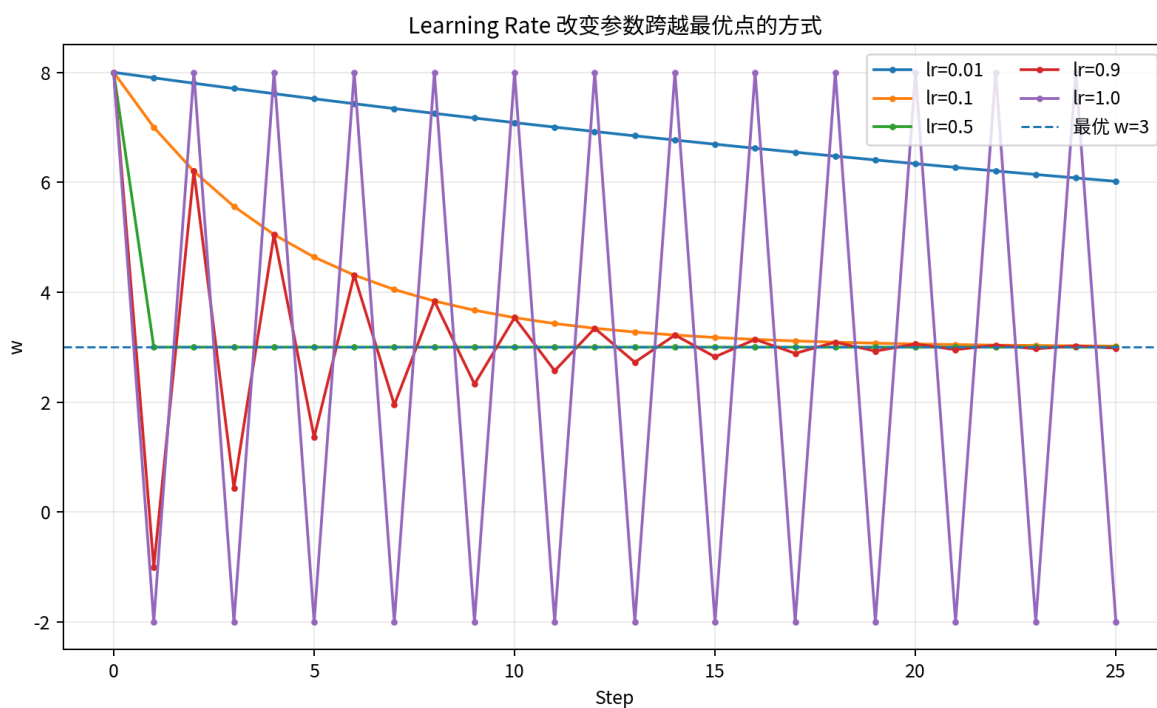
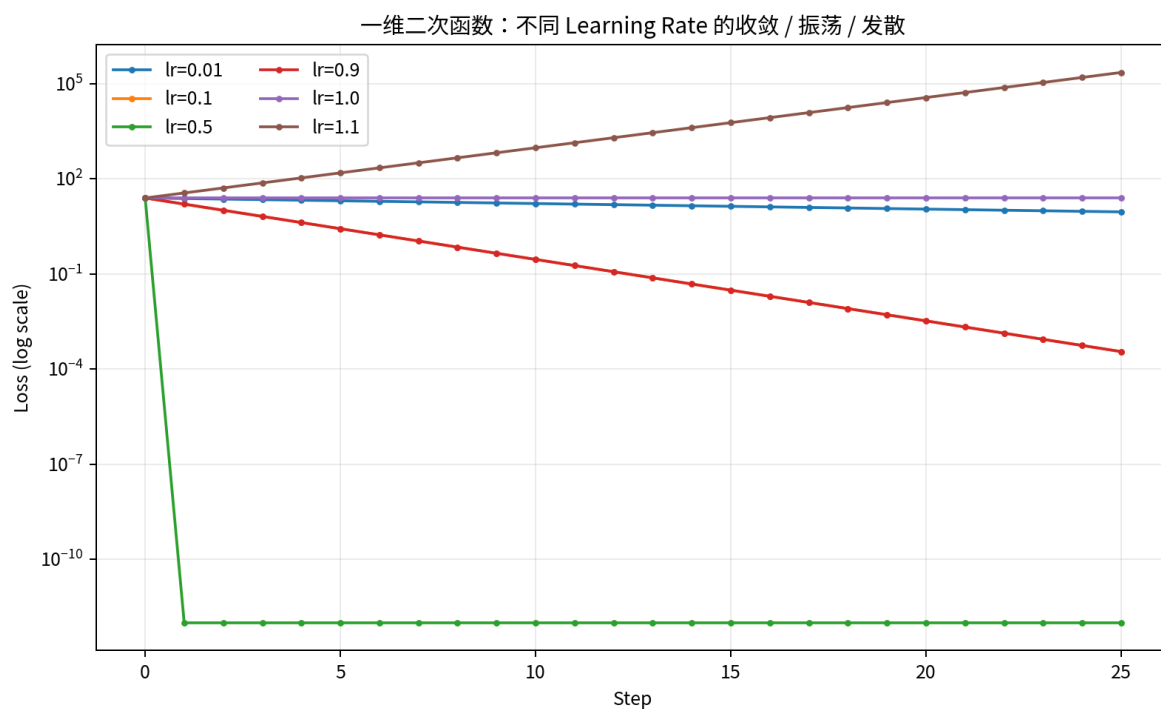
$$|1 - 2\eta| < 1$$

解得：

$$0 < \eta < 1$$

这不是所有神经网络的“通用 LR 上限”，只是这个特定二次函数的精确结果。但它非常适合建立直觉。

η	误差倍率 $1 - 2\eta$	行为
0.01	0.98	同方向慢慢靠近，极慢
0.1	0.8	稳定、较快
0.5	0	本例一步到最优点
0.9	-0.8	每步跨到另一侧，振荡但幅度缩小
1.0	-1	等幅振荡，永远不收敛
1.1	-1.2	振荡且幅度越来越大，发散



非常重要的底层直觉

“学习率大”并不是简单地把“慢速下降”变成“快速下降”。随着学习率增加，系统可能经历：

太慢 → 快速收敛 → 跨越最低点并振荡 → 不收敛 → 发散

这就是为什么调 LR 不是单调问题。

5 为什么同一个学习率在不同问题上表现完全不同？——曲率

现在考虑二维 Loss：

$$L(w_1, w_2) = \frac{1}{2} (w_1^2 + 20w_2^2)$$

Gradient：

$$\nabla L = \begin{bmatrix} w_1 \\ 20w_2 \end{bmatrix}$$

w_1 方向比较平缓，而 w_2 方向非常陡。更新 w_2 ：

$$w_{2,t+1} = (1 - 20\eta)w_{2,t}$$

要稳定，需要：

$$|1 - 20\eta| < 1$$

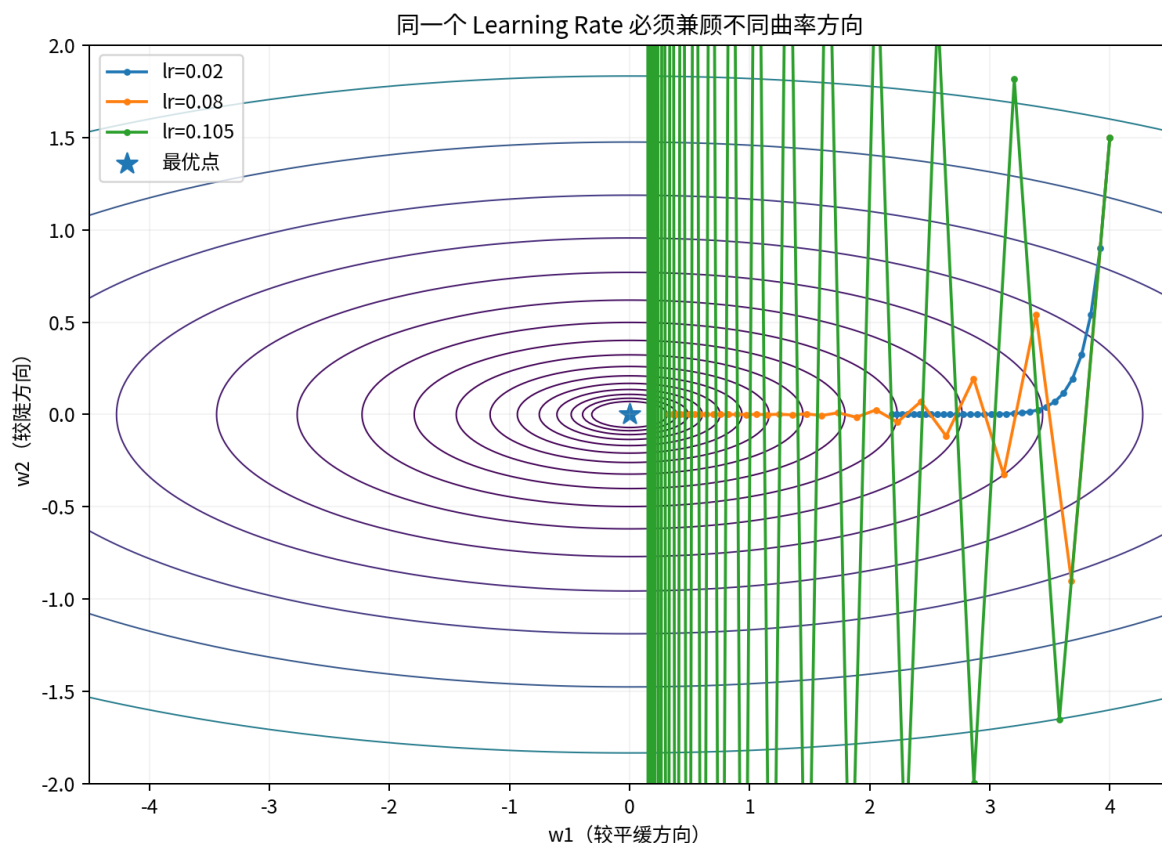
所以：

$$0 < \eta < 0.1$$

虽然 w_1 方向本来可以接受更大的步长，但一个**全局统一的 LR** 还必须照顾最陡的方向。

实验中：

- $\eta = 0.08$ 能稳定下降；
- $\eta = 0.095$ 接近稳定边界，陡方向衰减较慢并带振荡；
- $\eta = 0.105$ 超过稳定边界，Loss 最终爆到数千。



原理

Learning Rate 的稳定范围与 Loss Landscape 的局部曲率密切相关。更陡、更高曲率的方向允许的稳步行长更小。这也是为什么真实深层网络的优化比一维小例子复杂得多：不同 Parameter、不同层、不同方向的局部几何可能差异巨大。

6 Learning Rate、Gradient Norm 与 Update Norm 不是一回事

对于最简单 SGD:

$$\Delta\theta = -\eta g$$

因此:

$$\|\Delta\theta\| = \eta \|g\|$$

这里三个不同的量:

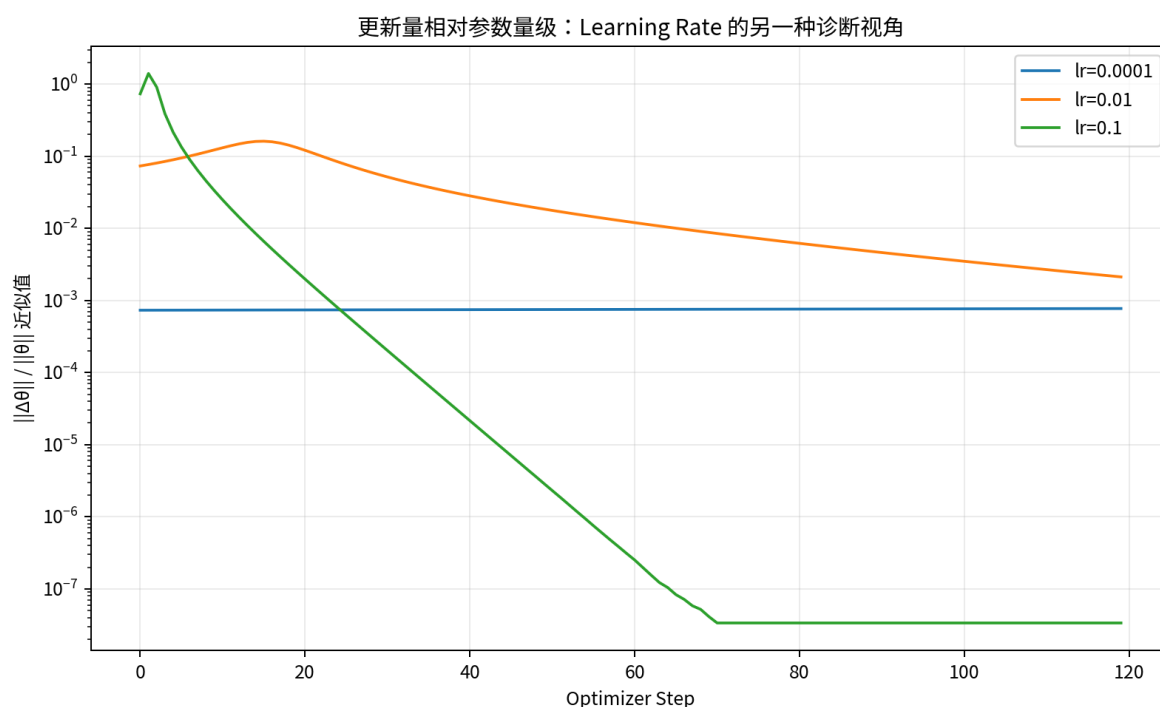
1. **Gradient Norm** $\|g\|$: 当前梯度有多大;
2. **Learning Rate** η : 更新规则的缩放系数;
3. **Update Norm** $\|\Delta\theta\|$: 参数这一步真正改变了多少。

因此即使 LR 不变, Gradient Norm 变大时, SGD 的实际 Parameter Update 也会变大。

有时还可以观察:

$$\frac{\|\Delta\theta\|}{\|\theta\|}$$

把更新量与当前参数量级作比较。它不是万能阈值，但能帮助识别“这一步相对参数本身是否异常巨大”。



常见误区

不要把“learning rate = 10^{-4} ”理解成“每个参数每次只改变 10^{-4} ”。实际更新量还受到 Gradient、Optimizer 内部状态、Weight Decay、Gradient Clipping 等因素影响。

7 PyTorch 实验：只改 LR，训练轨迹就完全不同

为了公平比较，本讲实验固定：

- 同一数据；
- 同一模型结构；
- 同一参数初始化；
- 同一 Optimizer (SGD)；
- 同样的 120 个 optimizer steps；
- 唯一主要改变：Learning Rate。

模型学习：

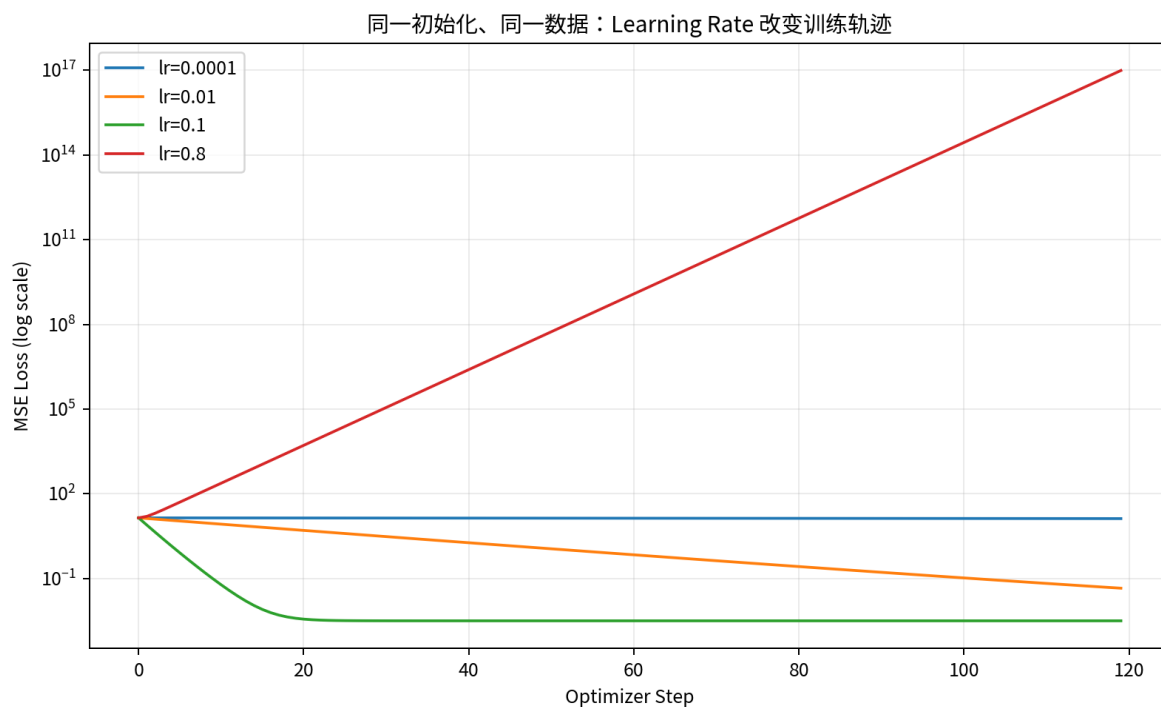
$$y = 2x + 1 + \text{small noise}$$

共同初始化：

$$w \approx -0.833, \quad b \approx -0.806$$

实验结果：

LR	最终 Loss	现象
10^{-4}	约 13.30	太小，120 步后仍几乎没学到
10^{-2}	约 0.0456	稳定下降，但仍未完全到最优附近
10^{-1}	约 0.00319	本实验中快速且稳定
0.8	约 9.8×10^{16}	参数爆炸，明显发散



与科研的关系

这个实验只能说明“在这套模型、数据、初始化、Optimizer、训练步数下，这些 LR 的优化行为不同”。不能据此推出“0.1 是神经网络最好的学习率”。科研中任何 LR 结论都必须带上具体训练设置。

8 Batch Size 与 Learning Rate：有关联，但没有机械万能公式

第 34 讲已经看到：小 Batch 的 Gradient Noise 更大，大 Batch 的 Gradient 更接近 Full-Batch Gradient。于是 LR 与 Batch Size 往往会相互影响。

一个常见经验是：更大 Batch 有时可以搭配更大 LR；某些大规模训练还会使用 linear scaling 等经验规则。但必须注意：

不要把经验法则当定律

“Batch 翻倍，LR 就必须翻倍”不是普适数学定律。可用 LR 还受到 Optimizer、模型、Normalization、Warmup、数据、Loss Landscape、训练阶段等影响。任何 scaling rule 都应视作起点或特定条件下的经验，而不是无需验证的结论。

科研比较 Batch Size 时，最好至少明确：

- LR 是否同时调整；

- scheduler 是否按相同 optimizer steps 或 tokens 对齐；
- global/effective batch 是否变化；
- 训练预算按 epochs、steps、samples 还是 tokens 对齐。

9 同样写 lr=0.001, SGD 和 Adam 为什么不能直接比较？

最简单 SGD：

$$\Delta\theta = -\eta g$$

所以 Update 与原始 Gradient 直接成比例。

Adam 则会维护一阶和二阶统计量，大致形成类似：

$$\Delta\theta \propto -\eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$

因此 nominal learning rate η 只是 Adam 更新中的一个全局缩放系数，真正每个参数的步长还会受到历史梯度统计影响。

实验从同一个点、同一个 Gradient 开始：

$$g = [8, -40]$$

当：

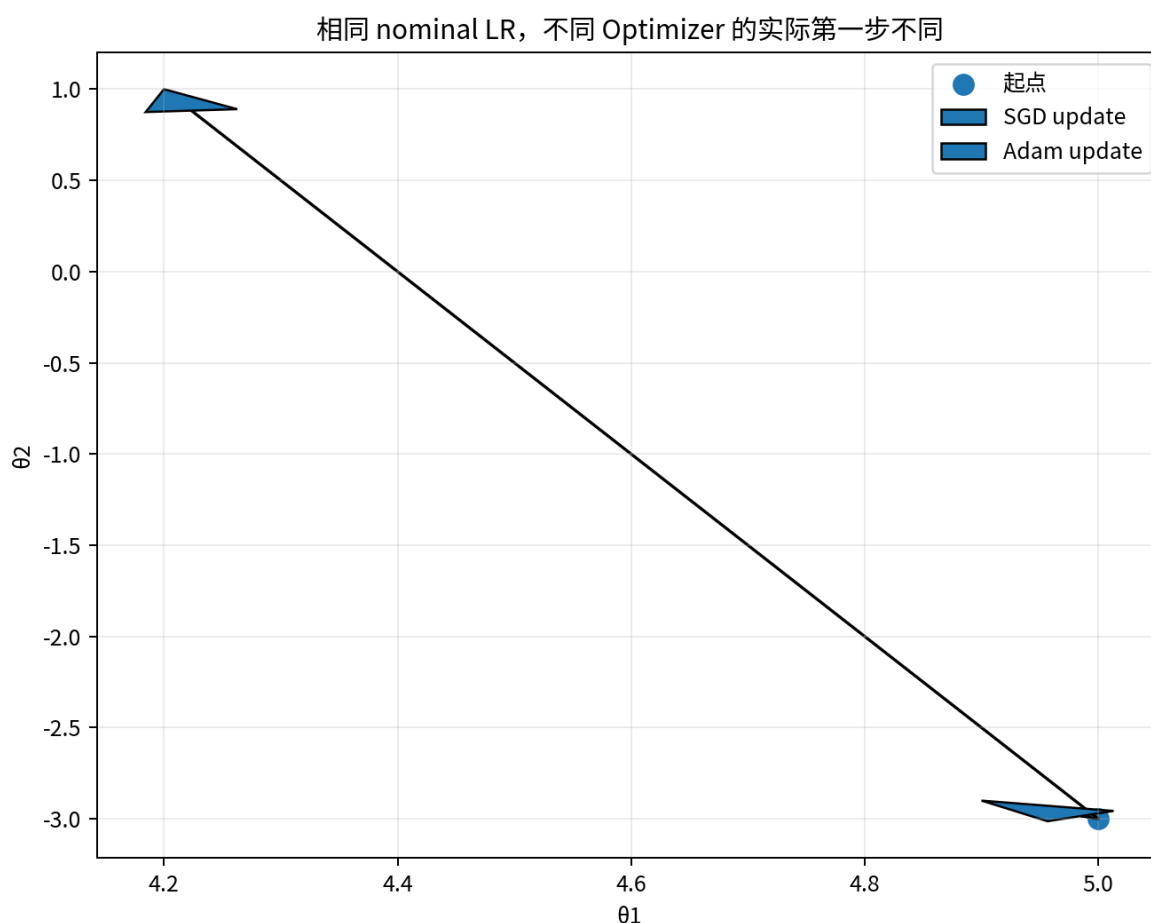
$$\eta = 0.1$$

第一步：

$$\Delta\theta_{SGD} \approx [-0.8, 4.0]$$

而 Adam 第一更新约：

$$\Delta\theta_{Adam} \approx [-0.1, 0.1]$$



原理

所以“Adam 用 3×10^{-4} ，SGD 用 0.1”不能简单得出“SGD 学习率大 333 倍”。两种 Optimizer 对 Gradient 的处理规则不同，Learning Rate 的数值不具有跨 Optimizer 的直接可比性。

10 Learning Rate Schedule: LR 不一定从头到尾固定

到目前为止，我们把 η 当常数。但真实训练常写成：

$$\eta_t = f(t)$$

也就是 Learning Rate 随 optimizer step 改变。

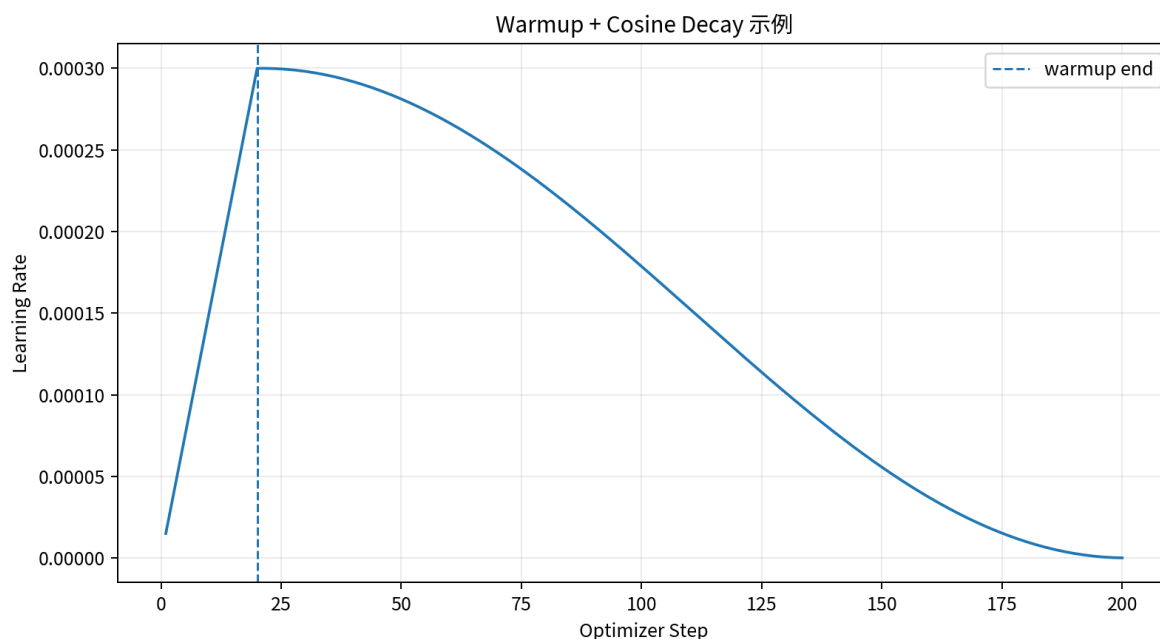
常见形式包括：

- Constant LR：始终不变；
- Step Decay：到指定阶段突然降低；
- Exponential Decay：持续指数衰减；
- Cosine Decay：按照余弦形状平滑下降；
- Warmup + Decay：前期从小 LR 升到 peak LR，再逐渐降低。

本讲示例：peak LR 为：

$$3 \times 10^{-4}$$

前 20 optimizer steps 进行线性 Warmup，然后做 Cosine Decay：



10.1 Warmup 到底是什么？

Warmup 的意思非常简单：训练刚开始先不用完整峰值 LR，而是从更小值逐步升高。

大规模 Transformer / LLM 训练中 Warmup 很常见，它可以帮助早期优化更加稳定。实际原因可能与初始参数状态、Optimizer moment 估计、Batch/Gradient 尺度、深层网络早期敏感性等共同相关。

常见误区

不要把 Warmup 理解成“前 1000 步不学习”。Warmup 阶段仍然在更新 Parameter，只是 LR 较小并逐步增大。

11 如何从训练曲线初步判断 LR？

下面只是**诊断线索**，不是看到某个形状就能百分百定罪。

11.1 可能太小

典型现象：

- Training Loss 长期缓慢下降；
- 参数更新量很小；
- 在固定训练预算内明显没有达到合理水平；
- 增大 LR 后训练速度显著改善且仍稳定。

11.2 可能太大

典型现象：

- Loss 剧烈震荡；
- Loss 突然爆炸到极大值或 NaN / Inf；
- Parameter / Update Norm 突然异常增大；
- 降低 LR 后问题显著缓解。

11.3 看起来合适

典型现象：

- Loss 在合理预算内明显下降；
- 没有持续发散；
- 参数更新不是长期接近零，也不是异常巨大；
- Validation 指标最终也需要单独检查。

11.4 Loss 不降，不一定是 LR

也可能是：

- 数据 / Label 错；
- Loss 实现错误；
- 忘了 `optimizer.step()`；
- 梯度被 detach；
- ReLU 大量关闭；
- 梯度消失；
- 参数没有注册进 Optimizer；
- Batch / dtype / device / mask 错误。

调试顺序

如果 Loss 不正常，不要第一反应就“疯狂换 LR”。先确认训练闭环：

data → forward → loss → backward → grad → step

每一步都真的成立，再讨论超参数。

12 PyTorch 代码：Learning Rate 到底在哪里？

最基本写法：

```
model = MyModel()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

```
for x, y in loader:
    optimizer.zero_grad()
    pred = model(x)
    loss = loss_fn(pred, y)
    loss.backward()
    optimizer.step()
```

这里：

- `loss.backward()` 负责算 Gradient；
- `lr=0.01` 是 Optimizer 更新规则中的学习率设置；
- `optimizer.step()` 才真正按照当前 LR 与 Gradient 更新参数。

如果 Scheduler 修改 LR：

```
optimizer.step()
scheduler.step()
```

下一次 Parameter Update 使用的 LR 可能已经发生变化。

所以日志里最好记录：

- optimizer step；
- 当前 LR；
- training loss；
- 必要时 gradient norm / update norm；
- validation metric。

13 与 LLM / Transformer 的直接关系

大型 Transformer 训练配置里经常看到：

```
optimizer = AdamW
peak_lr = 3e-4
warmup_steps = 2000
scheduler = cosine
weight_decay = 0.1
global_batch_size = ...
```

现在你应该知道：

- peak LR 是 schedule 中的峰值，而不一定从第 1 步就使用；
- warmup steps 应问清楚是不是 optimizer steps；
- 改变 Gradient Accumulation / Global Batch 后，optimizer step 数与 schedule 时间轴也可能变化；
- 同一个 peak LR 在不同 Optimizer、不同 Batch、不同模型规模上不能机械比较；
- Next-token Cross Entropy 是否稳定下降，与 LR 只是训练系统中的一个因素。

以后读 LLM 论文要问什么？

看到“ $LR=3 \times 10^{-4}$ ”时，不要只抄这个数字。至少继续找：Optimizer、Warmup、Decay、Batch / Tokens per Update、Total Steps / Tokens、Weight Decay、Gradient Clipping、Precision、模型规模与训练阶段。脱离这些上下文，单个 LR 数字几乎没有复现意义。

14 与 Agent Systems 科研的关系

你的长期研究对象是 Agent Systems，很多实验不一定训练基础模型，但仍可能遇到 LR：

- LoRA / QLoRA 微调 Agent backbone；
- 训练 Router、Critic、Verifier、Reward Model、Policy Head；
- 微调 Tool-use / planning / memory 模块；
- 多 Agent 数据上的 supervised fine-tuning；
- preference optimization 或 reinforcement-learning-like 更新。

这些场景的数据往往更少、更异质，有时 trajectory 内相关性很强。此时一个不合适的 LR 可能让结果对随机种子、数据顺序和初始化更加敏感。

与科研的关系

如果论文的核心现象可能被“某个 LR 恰好调得更好”解释，那么必须做合理的 LR 控制或敏感性分析。尤其比较两种方法时，不要只把同一个 LR 生硬复制给结构完全不同的训练配置，然后把差异全部解释成方法贡献。

15 科研中怎样设计 Learning Rate 对照实验？

如果目标是研究 LR 本身，建议至少固定：

- Dataset / split；
- 模型结构；
- 初始化或 Random Seed；
- Batch Size 与数据顺序；

- Optimizer;
- 训练步数 / tokens / compute budget;
- 其他 Regularization 与 Scheduler 条件。

然后改变 LR，例如对数尺度：

$$10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}$$

而不是只试非常接近的两个数。

记录：

- 是否发散；
- 多快达到某个 Training Loss；
- 最终 Training Loss；
- Validation / Test 指标；
- Gradient / Update Norm；
- 多随机种子下稳定性。

常见误区

“某 LR 的 Training Loss 最低”不等于“它的 Generalization 最好”。优化效果与泛化效果是两个相关但不同的问题，后面第 41–42 讲会专门处理。

16 本讲知识重新接回此前课程

你以前在 LLM 课程和本课前面已经见过：

Gradient → Optimizer → Parameter Update

现在进一步补全为：

Gradient：给出局部方向和尺度

↓

Learning Rate：控制全局步长尺度

↓

Optimizer：决定怎样加工 Gradient

↓

Scheduler：决定 LR 随时间怎样变化

这四者不能混为同一个概念。

17 本讲最低闭环能力

学完本讲后，至少应该做到：

1. 能解释 $\theta \leftarrow \theta - \eta g$ 每一项；
2. 知道 LR 太小会慢，太大可能振荡甚至发散；
3. 知道稳定 LR 与 Loss Landscape 曲率有关；
4. 知道 LR 数值不能脱离 Optimizer / Batch / Schedule 比较；
5. 能在 PyTorch 中找到当前 LR，并知道它什么时候影响 Parameter Update；
6. 看到 LLM 的 peak LR / warmup / cosine 不再把它们当作孤立名词；
7. 做科研对照时，知道必须控制初始化、训练预算、Batch、Optimizer 等变量。

一句话总结

Learning Rate 不是“学习多少知识”的比例，而是优化算法根据当前梯度信息更新参数时使用的步长尺度。合适的 LR 要在“足够快”和“仍处于稳定可控的局部更新范围”之间取得平衡。